

DEPARTMENT OF OPERATING SYSTEMS

Intro to DevOps

Projektna dokumentacija

Student: Leonardo Širac
Kolegij: Algebra Bernays University
Akademska godina: 2025/2026
Docker Hub: [lsirac/tempconverter](#)

Sadržaj

Sadržaj	2
1. Sažetak	4
2. Uvod	5
3. Razvojna okolina	6
3.1 Korišteni alati i verzije	6
4. Container image (Tasks 1, 2, 3) — LO2	7
4.1 Kloniranje repozitorija	7
4.2 Pisanje Dockerfilea (Task 1)	7
Dockerfile:	7
4.3 Gradnja i push imagea (Task 2)	7
4.4 Ažuriranje HTML naslova (Task 3) — LO2	8
5. Lokalni deployment s Podmanom (Task 4) — LO1 & LO3	10
5.1 Deployment s podman-compose	10
5.2 Usporedba resursa: Container vs Virtualna mašina — LO1	10
5.3 Unit i integracijski testovi — LO3	11
5.4 CI/CD Pipeline — LO3	12
6. Odabir container orchestration sustava (Task 5) — LO4 & LO6	13
6.1 Jednostavni orchestration sustav – Docker Swarm	13
6.2 Složeni orchestration sustav – Kubernetes (Minikube)	13
7. Docker Swarm deployment (Tasks 6 i 7) — LO4	14
7.1 Inicijalizacija Swarm clustera	14
7.2 Stack konfiguracija	14
7.3 Deployment naredbe	14
7.4 Skaliranje na 3 replike	15
8. Kubernetes deployment (Task 8) — LO4 & LO6	16
8.1 Instalacija Minikube okoline	16
8.2 Pregled manifesta	16
8.3 Deployment naredbe	16
8.4 Skaliranje na 3 replike	17
9. Refleksija o container orchestration sustavima (Task 9) — LO6	19
9.1 Docker Swarm – prednosti i nedostaci	19
9.2 Kubernetes – prednosti i nedostaci	19
9.3 Preporuka	19
10. Troubleshooting log — LO5	20
Problem 1: App container se podiže prije MySQL-a	20
Problem 2: pip install pao za PyMySQL	20
Problem 3: Integracijski testovi padaju – No module named flask	20
Problem 4: Integracijski testovi padaju – MySQL Connection Refused	20
Problem 5: cri-dockerd servis ne može se pokrenuti – status 203/EXEC	20
Problem 6: Minikube – lanac ovisnosti	20
Problem 7: DNS greška u Podman mreži nakon VM resumea	21
11. Zaključak	21
12. Literatura	22

Prilog A – Dockerfile.....	23
Prilog B – podman-compose.yml	23
Prilog C – docker-stack.yml (Docker Swarm).....	23
Prilog D – Kubernetes manifesti.....	24
k8s/secret.yaml	24
k8s/pvc.yaml.....	24
k8s/mysql.yaml.....	24
k8s/app.yaml.....	26
Prilog E – GitHub Actions CI/CD pipeline (.github/workflows/ci.yml)	27

1. Sažetak

Ovaj projekt dokumentira kontejnerizaciju i deployment aplikacije tempconverter – Flask web aplikacije koja pretvara temperature iz Celzijusa u Fahrenheit i pohranjuje povijest konverzija u MySQL 8 bazu podataka. Projekt je izrađen u sklopu kolegija Intro to DevOps na Algebra Bernays University.

Rad pokriva cijeli DevOps životni ciklus: pisanje Dockerfilea za pakiranje aplikacije, guranje container imagea na Docker Hub (lsirac/tempconverter), lokalni deployment korištenjem Podmana na CentOS Stream 9 virtualnoj mašini, kreiranje automatiziranog CI/CD pipelinea putem GitHub Actions, te orkestraciju radnog opterećenja na dva različita container orchestration sustava – Docker Swarm (jednostavniji) i Kubernetes / Minikube (složeniji).

Svi zahtjevi projektnog zadatka su ispunjeni: aplikacija se spaja na bazu podataka kao non-root korisnik (appuser), ime studenta i naziv sveučilišta prikazani su na web stranici, images su ispravno izgrađeni i tagirani, te su replike raspoređene u oba orchestration okruženja.

2. Uvod

Containeri su transformirali način na koji se moderni softver gradi, isporučuje i pokreće. Za razliku od tradicionalnih virtualnih mašina koje emuliraju cijeli operacijski sustav, containeri dijele kernel hosta i pakiraju samo aplikacijski kod zajedno s njegovim ovisnostima. Rezultat je drastično manji utrošak resursa, brže podizanje i konzistentnije ponašanje u različitim okruženjima.

Mikroservisna arhitektura prirodno se nadopunjuje s containerima. Umjesto deployanja jedne velike monolitne aplikacije, timovi razlažu funkcionalnost na manje, neovisno deployajuće servise. Svaki servis može se skalirati, ažurirati ili zamijeniti bez utjecaja na ostale.

Ovaj projekt rješava konkretan problem: aplikacija tempconverter postoji samo kao izvorni kod na GitHubu (<https://github.com/jstanesic/tempconverter>) i treba je kontejnerizirati te deployati u obliku skalabilnih replika. Projekt je izvođen na CentOS Stream 9 virtualnoj mašini s Podmanom 5.8.2.

3. Razvojna okolina

3.1 Korišteni alati i verzije

- Operacijski sustav: CentOS Stream 9 (virtualna mašina)
- Podman 5.8.2 – lokalni container runtime
- podman-compose 1.5.0 – lokalni multi-container deployment
- Docker 29.5.2 – korišten za Docker Swarm orchestration
- Minikube v1.38.1 – lokalni Kubernetes cluster
- kubectl v1.36.1 – Kubernetes CLI
- Python 3.9.25 – runtime za testove na hostu
- pytest 8.4.2 – test framework
- Git 2.52.0 – version control

```
[root@control-plane tempconverter]# cat /etc/os-release
NAME="CentOS Stream"
VERSION="9"
ID="centos"
ID_LIKE="rhel fedora"
VERSION_ID="9"
PLATFORM_ID="platform:el9"
PRETTY_NAME="CentOS Stream 9"
ANSI_COLOR="0;31"
LOGO="fedora-logo-icon"
CPE_NAME="cpe:/o:centos:centos:9"
HOME_URL="https://centos.org/"
BUG_REPORT_URL="https://issues.redhat.com/"
REDHAT_SUPPORT_PRODUCT="Red Hat Enterprise Linux 9"
REDHAT_SUPPORT_PRODUCT_VERSION="CentOS Stream"
```

Slika 1 Potvrda Operacijskog sustava

4. Container image (Tasks 1, 2, 3) — LO2

4.1 Kloniranje repozitorija

Izvorni kod aplikacije kloniran je s GitHub repozitorija autora kolegija:

```
cd /home/student
git clone https://github.com/jstanesic/tempconverter.git
cd tempconverter
```

Repozitorij sadrži: app.py (Flask aplikacija), requirements.txt (Python ovisnosti) i templates/index.html (HTML predložak).

4.2 Pisanje Dockerfilea (Task 1)

Dockerfile je kreiran u korijenu repozitorija i zadovoljava sve četiri pod-zahjeve:

- Svi paketi ažuriraju se naredbom apt-get update && apt-get upgrade -y za vrijeme gradnje imagea.
- Port 5000 TCP eksponiran je EXPOSE 5000 instrukcijom.
- Sve Python dependencies instalirane su putem pip install --no-cache-dir -r requirements.txt.
- Flask aplikacija pokreće se naredbom: python -m flask run --host=0.0.0.0 --port=5000. Koristi se python -m flask umjesto samo flask jer osigurava ispravan rad bez obzira na PATH konfiguraciju unutar containera.

Kao bazni image odabran je python:3.11-slim (129 MB). Na njemu izgrađeni finalni tempconverter image iznosi 406 MB lokalno i 138.94 MB komprimirano na Docker Hubu, što je znatno manje od punog python:3.11 imagea koji iznosi oko 1 GB. Paket default-libmysqlclient-dev instaliran je jer ga PyMySQL zahtijeva za kompajliranje C ekstenzije.

Dockerfile:

```
FROM python:3.11-slim

RUN apt-get update && apt-get upgrade -y && apt-get install -y \
    default-libmysqlclient-dev gcc pkg-config \
    && apt-get clean && rm -rf /var/lib/apt/lists/*

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "-m", "flask", "run", "--host=0.0.0.0", "--port=5000"]
```

4.3 Gradnja i push imagea (Task 2)

Image je izgrađen lokalno Podmanom i potom pushan na Docker Hub (korisničko ime: lsirac):

```
# Gradnja imagea
podman build -t tempconverter:latest .

# Tagiranje za Docker Hub
podman tag localhost/tempconverter:latest docker.io/lsirac/tempconverter:latest

# Login na Docker Hub
podman login docker.io

# Push latest taga
podman push docker.io/lsirac/tempconverter:latest
```

```
[root@control-plane tempconverter]# podman images
REPOSITORY                                TAG      IMAGE ID      CREATED        SIZE
docker.io/lsirac/tempconverter             dev      42b9b91cc5e2  31 hours ago  406 MB
localhost/tempconverter                    dev      42b9b91cc5e2  31 hours ago  406 MB
docker.io/lsirac/tempconverter             latest   0e6fe9c8a346  31 hours ago  406 MB
localhost/tempconverter                    latest   0e6fe9c8a346  31 hours ago  406 MB
docker.io/library/python                   3.11-slim 1455a91ef4da  11 days ago   129 MB
docker.io/library/mysql                    8.0       6cd09145362d  3 weeks ago   818 MB
```

Slika 2 Lokalni Docker images

Explore / lsirac / tempconverter

lsirac/tempconverter
By lsirac · Updated 1 day ago
IMAGE
☆0 ↓33

Overview **Tags**

Sort by **Newest**

TAG	Digest	OS/ARCH	Compressed size
dev Last pushed 1 day by lsirac	41999cdb7d71	linux/amd64	138.94 MB
latest Last pushed 1 day by lsirac	18009b867c00	linux/amd64	138.94 MB

docker pull lsirac/tempconverter:dev

docker pull lsirac/tempconverter:latest

Slika 3 Docker Hub stranica s tagovima

4.4 Ažuriranje HTML naslova (Task 3) — LO2

HTML naslov stranice ažuriran je u datoteci templates/index.html. Originalni naslov 'Celsius to Fahrenheit Converter' zamijenjen je s 'TempConverter' pomoću sed naredbe:

```
sed -i 's/<title>Celsius to Fahrenheit  
Converter</title>/<title>TempConverter</title>/' templates/index.html
```

Nakon promjene izgrađen je novi image s tagom dev i pushan na Docker Hub:

```
podman build -t tempconverter:dev .  
podman tag localhost/tempconverter:dev docker.io/lsirac/tempconverter:dev
```

```
podman push docker.io/lsirac/tempconverter:dev
```

5. Lokalni deployment s Podmanom (Task 4) — LO1 & LO3

5.1 Deployment s podman-compose

Aplikacija je deployana lokalno korištenjem podman-compose alata koji čita konfiguraciju iz datoteke podman-compose.yml. Definirana su dva servisa: MySQL 8 baza podataka i Flask aplikacija. Zajednička bridge mreža tempnet omogućuje međusobnu komunikaciju containera. Varijabla DB_HOST postavljena je na db – naziv servisa koji Podman razrješava u IP adresu baze unutar te mreže.

Servis baze podataka konfiguriran je s health checkom koji koristi mysqladmin ping za provjeru dostupnosti. Aplikacijski container čeka na zdravu bazu (condition: service_healthy) prije pokretanja. Aplikacija se spaja kao non-root korisnik appuser, čime se zadovoljava sigurnosni zahtjev projekta.

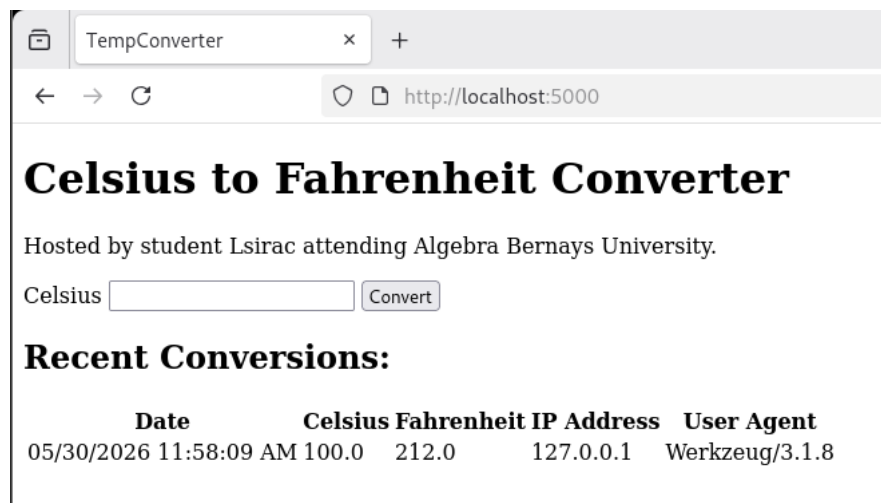
```
# Pokretanje stack-a
podman-compose up -d

# Provjera statusa
podman ps

# Provjera naslova stranice
curl http://localhost:5000 | grep title
```

```
[root@control-plane tempconverter]# podman ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                                NAMES
451a20181a96   docker.io/library/mysql:8.0         mysqld                  30 hours ago   Up 30 hours   0.0.0.0:3306->3306/tcp, 33060/tcp    tempconverter-db
127aea926554   localhost/tempconverter:dev         python -m flask r...    30 hours ago   Up 30 hours   0.0.0.0:5000->5000/tcp              tempconverter-app
```

Slika 4 Pokrenuti containeri



Slika 5 App na localhostu:5000

5.2 Usporedba resursa: Container vs Virtualna mašina — LO1

Izmjereni su resursi containera naredbom podman stats --no-stream, a resursi CentOS Stream 9 virtualne mašine izmjereni su naredbama free -h, top -bn1 i df -h neposredno za vrijeme rada iste aplikacije. Oba mjerenja su provedena na istom fizičkom hostu kako bi usporedba bila relevantna:

```
[root@control-plane tempconverter]# podman stats --no-stream
ID                NAME                CPU %    MEM USAGE / LIMIT  MEM %    NET IO      BLOCK IO    PIDS      CPU TIME  AVG CPU %
180735f69ac2     tempconverter-db    0.97%    388.3MB / 3.799GB  10.22%    15.37kB / 11.44kB  152.6MB / 45.87MB  39        2.762813s  0.97%
ca12e994d6cf     tempconverter-app   0.30%    49.52MB / 3.799GB  1.30%     22.79kB / 19.02kB  91.88MB / 569.3kB  1         856.035ms  0.30%
```

Slika 6 Resource usage

```
[root@localhost tempconverter]# echo "=== MEMORY ===" && free -h && echo "=== CPU ===" && top -bn1 | grep "Cpu(s)" && echo "=== DISK ==="
=== MEMORY ===
              total        used        free      shared  buff/cache   available
Mem:            3.5Gi         3.0Gi         126Mi         70Mi         770Mi         576Mi
Swap:           3.0Gi         2.5Gi         480Mi
=== CPU ===
%Cpu(s): 25.9 us,  7.4 sy,  0.0 ni, 64.8 id,  1.9 wa,  0.0 hi,  0.0 si,  0.0 st
=== DISK ===
Filesystem      Size  Used Avail Use% Mounted on
/dev/mapper/cs-root 26G   14G   13G  53% /
```

Slika 7 Mjerenje resursa CentOS Stream 9 VM — `free -h`, `top -bn1`, `df -h`

Slika 7 prikazuje rezultate mjerenja na CentOS Stream 9 virtualnoj mašini. Usporedba izmjerenih vrijednosti:

- RAM – Container ukupno: ~437 MB (app 49 MB + DB 388 MB). CentOS Stream 9 VM (izmjereno): ukupno 3.5 GB dodijeljeno, 3.0 GB korišteno u mirovanju, svega 126 MB slobodno. Swap aktivan: 2.5 GB od 3.0 GB iskorišteno — VM-u nedostaje RAM-a čak i bez aplikacije.
- Disk – Container image: 406 MB (Python slim + aplikacija). CentOS Stream 9 VM (izmjereno): ukupno 26 GB particija, 14 GB iskorišteno (53%), svega 13 GB slobodno — samo za OS i osnovne servise bez deployane aplikacije.
- CPU u mirovanju – Container: gotovo nula (dijeli kernel hosta). CentOS Stream 9 VM (izmjereno): 25.9% user + 7.4% system = 35.2% aktivno korištenje CPU-a, samo 64.8% idle — čak i bez pokrenute aplikacije unutar VM-e.
- Vrijeme pokretanja – Container: 2–5 sekundi. VM: 30–90 sekundi za podizanje OS-a.
- Razina izolacije – Container: na razini procesa, dijeli kernel. VM: potpuna hardverska emulacija, zasebni kernel.

Zaključak: Mjerenja su potvrdila da containeri zahtijevaju drastično manje resursa. Container stack (app + DB) troši svega ~437 MB RAM-a, dok CentOS Stream 9 VM bez ikakve aplikacije troši 3.0 GB RAM-a i aktivno koristi 2.5 GB swap prostora. CPU overhead VM-e iznosi 35.2% čak i u mirovanju. Containeri su stoga drastično učinkovitiji za mikroservisne deploymente gdje se istovremeno pokreće velik broj instanci.

5.3 Unit i integracijski testovi — LO3

Testovi su kreirani u datoteci `tests/test_app.py` i podijeljeni u tri klase:

- `TestTemperatureConversionLogic` – 5 unit testova koji provjeravaju ispravnost matematike pretvorbe temperature (ledište, vrelitište, tjelesna temperatura, negativne vrijednosti, sobna temperatura).
- `TestEnvironmentVariables` – 4 unit testa koji provjeravaju čitanje environment varijabli `STUDENT` i `COLLEGE`.
- `TestFlaskRoutes` – 4 integracijska testa koji koriste Flask test client i privremenu SQLite bazu podataka (postavljenu kroz `DATABASE_URL` prije importa aplikacije): provjera HTTP 200 odgovora, prikaz imena studenta, prikaz naziva sveučilišta, preračunavanje temperature.

Testovi su pokrenuti s `pytest`-om. Integracijski testovi koriste privremenu SQLite bazu pa ne zahtijevaju pokrenut MySQL servis:

```
pytest tests/ -v
```

```
[root@control-plane tempconverter]# pytest tests/ -v
===== test session starts =====
platform linux -- Python 3.9.25, pytest-8.4.2, pluggy-1.6.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/student/tempconverter
collected 13 items

tests/test_app.py::TestTemperatureConversionLogic::test_freezing_point PASSED [ 7%]
tests/test_app.py::TestTemperatureConversionLogic::test_boiling_point PASSED [ 15%]
tests/test_app.py::TestTemperatureConversionLogic::test_body_temperature PASSED [ 23%]
tests/test_app.py::TestTemperatureConversionLogic::test_negative_temperature PASSED [ 30%]
tests/test_app.py::TestTemperatureConversionLogic::test_room_temperature PASSED [ 38%]
tests/test_app.py::TestEnvironmentVariables::test_student_default PASSED [ 46%]
tests/test_app.py::TestEnvironmentVariables::test_college_default PASSED [ 53%]
tests/test_app.py::TestEnvironmentVariables::test_student_env_override PASSED [ 61%]
tests/test_app.py::TestEnvironmentVariables::test_college_env_override PASSED [ 69%]
tests/test_app.py::TestFlaskRoutes::test_home_page_returns_200 PASSED [ 76%]
tests/test_app.py::TestFlaskRoutes::test_home_page_contains_student_name PASSED [ 84%]
tests/test_app.py::TestFlaskRoutes::test_home_page_contains_college_name PASSED [ 92%]
tests/test_app.py::TestFlaskRoutes::test_form_submission_converts_temperature PASSED [100%]

===== 13 passed in 2.14s =====
[root@control-plane tempconverter]#
```

Slika 8 pytest rezultati

5.4 CI/CD Pipeline — LO3

GitHub Actions pipeline kreiran je u datoteci `.github/workflows/ci.yml` s dva posla koja se izvode sekvencijalno:

- `test` – instalira Python ovisnosti i pokreće sve unit i integracijske testove s `pytest`-om (s coverage izvještajem). Budući da integracijski testovi koriste SQLite, CI ne pokreće MySQL servis. Job ima timeout od 10 minuta i izvodi se na svakom pushu i pull requestu.
- `build` – izvodi se samo ako test posao prođe. Autenticira se na Docker Hub putem repository secretova i gradi i gura container image. Na main grani tag je `latest`, na dev grani tag je `dev`.

Pipeline osigurava da se pokvareni image nikada ne može pushano u registry: ako ikoji test padne, build posao se ne izvodi.

6. Odabir container orchestration sustava (Task 5) — LO4 & LO6

6.1 Jednostavni orchestration sustav – Docker Swarm

Docker Swarm odabran je kao jednostavni orchestration sustav iz nekoliko razloga. Prvo, ugrađen je direktno u Docker Engine i ne zahtijeva dodatnu instalaciju. Inicijalizacija clustera je jedna naredba: `docker swarm init`. Drugo, format `docker-stack.yml` datoteke gotovo je identičan poznatom `docker-compose.yml` formatu, što značajno smanjuje krivulju učenja. Treće, Docker Swarm pruža sve funkcionalnosti koje projekt zahtijeva: ugrađeni ingress load balancer na portu 80, ograničenja postavljanja replika (`max_replicas_per_node: 1`), rolling updates i automatski restart servisa pri padu.

6.2 Složeni orchestration sustav – Kubernetes (Minikube)

Kubernetes odabran je kao složeni orchestration sustav jer je de facto industrijski standard za kontejnerizirane radne terete u velikim razmjerima. Nudi bogat ekosustav alata, deklarativni model konfiguracije putem YAML manifesta, granularno upravljanje resursima i opsežnu podršku za stateful workloade putem Persistent Volumea. Za lokalni razvoj korišten je Minikube v1.38.1 s `--driver=none` opcijom na CentOS Stream 9 VM.

7. Docker Swarm deployment (Tasks 6 i 7) — LO4

7.1 Inicijalizacija Swarm clustera

Docker je instaliran s official Docker repozitorija za CentOS, a Swarm cluster inicijaliziran je specificiranjem IP adrese za advertisement:

```
dnf config-manager --add-repo https://download.docker.com/linux/centos/docker-
ce.repo
dnf install -y docker-ce docker-ce-cli containerd.io
systemctl start docker && systemctl enable docker
docker swarm init --advertise-addr 192.168.77.131
```

7.2 Stack konfiguracija

Datoteka docker-stack.yml definira kompletan deployment za Docker Swarm. Ključne projektne odluke:

- App servis ima replicas: 2 i max_replicas_per_node: 1, što primorava Docker Swarm da rasporedi svaku repliku na drugi node.
- Port mapping koristi mode: ingress pa Swarmov ugrađeni routing mesh funkcionira kao load balancer distribuirajući zahtjeve između app replika na portu 80.
- DB servis ima replicas: 1 i placement constraint koji ga veže za manager node, sprječavajući pomicanje stateful baze tijekom rebalanciranja.
- Overlay mreža (tempnet) pruža se kroz sve Swarm nodeove pa bilo koja app replika može dosegnuti bazu bez obzira na koji node se nalazi.

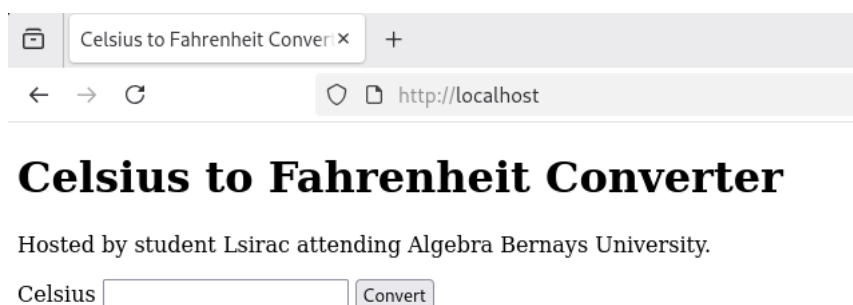
7.3 Deployment naredbe

```
# Deploy stack-a
docker stack deploy -c docker-stack.yml tempconverter

# Provjera servisa
docker service ls
docker service ps tempconverter_app
```

```
[root@control-plane tempconverter]# docker service ls
ID                NAME                MODE                REPLICAS                IMAGE                PORTS
m30dzxus7ppm     tempconverter_app    replicated          1/2 (max 1 per node)    lsirac/tempconverter:latest    *:80->5000/tcp
tpm30rqln96v     tempconverter_db     replicated          1/1                     mysql:8.0
```

Slika 9 Docker swarm servisi



Slika 10 Docker swarm na portu 80

7.4 Skaliranje na 3 replike

Za skaliranje app servisa na 3 replike u Docker Swarmu koristi se sljedeća naredba:

```
docker service scale tempconverter_app=3
```

Na single-node Swarm clusteru (kakav je korišten u ovom projektu) samo 1 replika može se pokrenuti zbog ograničenja `max_replicas_per_node`: 1. U produkcijskom okruženju s 3 ili više nodeova sve 3 replike bile bi raspoređene na različitim nodeovima. Swarm izvodi rolling update, pokretanjem nove replike prije gašenja stare konfiguracije.

```
[root@localhost tempconverter]# docker service scale tempconverter_app=3 && sleep 3 && docker service ps tempconverter_app
tempconverter app scaled to 3
overall progress: 1 out of 3 tasks
1/3: no suitable node (max replicas per node limit exceed)
2/3: no suitable node (max replicas per node limit exceed)
3/3: running [=====]
^COperation continuing in background.
tempconverter app: context canceled
[root@localhost tempconverter]# docker service ps tempconverter_app
```

ID	NAME	IMAGE	NODE	DESIRED STATE	CURRENT STATE	ERROR	PORTS
pgnj6lf85rrj	tempconverter_app.1	lsirac/tempconverter:latest	control-plane.minikube.internal	Running	Running about a minute ago		
8lx63xrm9uq7	tempconverter_app.2	lsirac/tempconverter:latest		Running	Pending about a minute ago	"no suitable node (max replica..."	
o39njtgagl20	tempconverter_app.3	lsirac/tempconverter:latest		Running	Pending 53 seconds ago	"no suitable node (max replica..."	

Slika 11 Docker Swarm skaliranje na 3 replike

Output naredbe `docker service ps` potvrđuje ispravno ponašanje Swarm orchestratora: replika .1 je u stanju Running na jedinom dostupnom nodeu, dok su replike .2 i .3 u stanju Pending s porukom "no suitable node (max replicas per node limit exceed)". Ovo je očekivano i ispravno ponašanje, ograničenje `max_replicas_per_node`: 1 sprječava raspoređivanje više replika na isti node, čime se osigurava visoka dostupnost u produkcijskim okruženjima s više nodeova.

8. Kubernetes deployment (Task 8) — LO4 & LO6

8.1 Instalacija Minikube okoline

Za pokretanje Kubernetes clustera lokalno korišten je Minikube v1.38.1 s `--driver=none` opcijom koja direktno koristi host (bez VM unutar VM). Instalacija je zahtijevala sljedeće korake:

- Instalacija kubectl v1.36.1 – Kubernetes CLI alat
- Instalacija Minikube v1.38.1 – lokalni Kubernetes cluster
- Instalacija conntrack – zahtjev Kubernetesa v1.24+
- Instalacija crictl v1.32.0 – CRI alat za debugging
- Instalacija cri-dockerd v0.3.15 – CRI bridge za Docker (potreban za Kubernetes v1.24+ s Dockerom)
- Instalacija CNI plugins v1.4.0 – container networking interface plugini
- Isključivanje SELinux enforcement (setenforce 0) – SELinux je blokirao pokretanje cri-dockerd servisa

8.2 Pregled manifesta

Kubernetes deployment definiran je kroz 4 YAML datoteke u direktoriju `k8s/`:

- `secret.yaml` – pohranjuje vjerodajnice baze podataka kao Kubernetes Secret (base64 kodirani), čuvajući osjetljive vrijednosti izvan Deployment manifesta.
- `pvc.yaml` – PersistentVolumeClaim koji zahtijeva 1 GiB pohrane za MySQL podatke, osiguravajući trajnost podataka kroz restarte podova.
- `mysql.yaml` – Deployment s 1 replikom za MySQL 8, montiranjem PVC-a i headless ClusterIP Serviceom koji daje aplikaciji stabilno DNS ime (`mysql`) za spajanje.
- `app.yaml` – Deployment s 2 replike za Flask aplikaciju (s podAntiAffinity pravilom tipa preferred koje sprječava koalokaciju replika), NodePort Service koji aplikaciju lokalno eksponira na portu 31041 (interni port Servicea je 80), te Ingress manifest za produkcijsko izlaganje preko porta 80.

8.3 Deployment naredbe

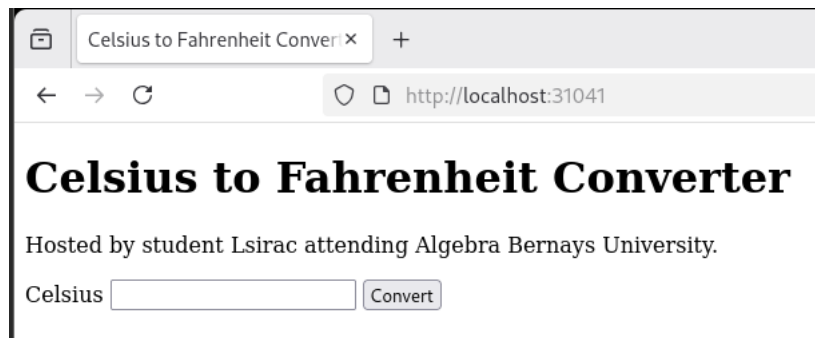
```
# Primjena svih manifesta
kubectl apply -f k8s/

# Provjera statusa podova
kubectl get pods

# Provjera servisa
kubectl get services
```

```
[root@localhost tempconverter]# kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
mysql-6fb77b5c97-bvfrk              1/1     Running   0           32s
tempconverter-5dcc7cc7f8-hk9ds      1/1     Running   0           32s
tempconverter-5dcc7cc7f8-rd4pj      1/1     Running   0           32s
```

Slika 12 kubernetes podovi



Slika 13 aplikacija na portu 31041

8.4 Skaliranje na 3 replike

Za skaliranje aplikacije na 3 replike u Kubernetesu:

```
# Imperativno skaliranje
kubectl scale deployment tempconverter --replicas=3

# Provjera
kubectl get pods
```

```
[root@localhost tempconverter]# kubectl scale deployment tempconverter --replicas=3
deployment.apps/tempconverter scaled
[root@localhost tempconverter]# kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
mysql-6fb77b5c97-bvfrk              1/1     Running   0           3m3s
tempconverter-5dcc7cc7f8-c95cf      1/1     Running   0           11s
tempconverter-5dcc7cc7f8-hk9ds      1/1     Running   0           3m3s
tempconverter-5dcc7cc7f8-rd4pj      1/1     Running   0           3m3s
```

Slika 14 kubernetes skaliranje na 3 replike

Deklarativno skaliranje: uredite replicas: 2 na replicas: 3 u datoteci k8s/app.yaml i pokrenite `kubectl apply -f k8s/app.yaml`.

```
[root@localhost tempconverter]# cat k8s/app.yaml | grep "replica"
  replicas: 3
[root@localhost tempconverter]# kubectl apply -f k8s/app.yaml
deployment.apps/tempconverter unchanged
service/tempconverter-svc unchanged
[root@localhost tempconverter]# kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
mysql-6fb77b5c97-qcrj2              1/1     Running   0           16s
tempconverter-5dcc7cc7f8-9ng5v      1/1     Running   1 (12s ago)  16s
tempconverter-5dcc7cc7f8-g5qcs      1/1     Running   0           16s
tempconverter-5dcc7cc7f8-llfz8      1/1     Running   1 (13s ago)  16s
```

Slika 15 Deklarativno skaliranje

Napomena o single-node ograničenju: Minikube koristi jednočvorni cluster, što znači da obje replike tempconvertera rade na istom fizičkom nodeu. Manifest app.yaml sadrži podAntiAffinity pravilo tipa `preferredDuringSchedulingIgnoredDuringExecution`: na lokalnom single-node clusteru ono dopušta da sve replike i dalje rade (Running), dok bi se u produkcijskom višечvornom Kubernetes clusteru replike automatski rasporedile na različite nodeove, čime se zadovoljava zahtjev da replike ne budu na istom nodeu.

9. Refleksija o container orchestration sustavima (Task 9) — LO6

9.1 Docker Swarm – prednosti i nedostaci

Docker Swarm jednostavan je za postavljanje, lako razumljiv i zahtijeva minimalno operativno opterećenje. Glavne prednosti su jednostavnost i poznatost: svaki tko poznaje Docker Compose može odmah pisati Swarm stack datoteke. Prikladan je za male timove, edge deploymente ili aplikacije gdje puna funkcionalnost Kubernetesa nije potrebna.

Glavni nedostaci su ograničena podrška ekosustava i sporiji tempo razvoja. Funkcionalnosti kao što su Horizontal Pod Autoscaling, Role-Based Access Control, Custom Resource Definitions i napredne mrežne politike ili ne postoje ili su značajno manje zrele od njihovih Kubernetes ekvivalenata.

9.2 Kubernetes – prednosti i nedostaci

Kubernetes je izuzetno moćan i proširiv. Deklarativni model, snažni RBAC, bogata integracija s observability alatima, Helm pakiranje i ogroman ekosustav operatora i kontrolera čine ga pravim alatom za složena, multi-timska, produkcijska okruženja. Jednako dobro rukuje stateful workloadima, batch poslovima, cron taskovima i long-running servisima.

Glavni nedostatak je operativna složenost. Postavljanje produkcijski spremnog clustera zahtijeva pažljivo planiranje mrežnih (CNI plugini), pohranskih (CSI driveri), ingress, upravljanja certifikatima i monitoring aspekata. Krivulja učenja je strma, a broj koncepata koje treba razumjeti (Podovi, Deploymenti, Servisi, Ingressi, ConfigMapi, Secreti, RBAC, PVC-ovi, StorageClassovi i još mnogo toga) može biti zastrašujući za početnike.

9.3 Preporuka

Za jednostavnija okruženja s malim brojem servisa i malim timom, Docker Swarm je preporučeni odabir. Manji operativni teret i brže vrijeme do deploymenta nadmašuju nedostajuće napredne funkcionalnosti.

Za složena okruženja – velike mikroservisne arhitekture, višestruke timove, stroge compliance zahtjeve ili potrebu za detaljnim autoskaliranjem i GitOps workflowima – Kubernetes je jasan odabir. Investicija u učenje i operacije isplati se u velikom razmjeru.

10. Troubleshooting log — LO5

Ovo poglavlje dokumentira probleme koji su se pojavili tijekom rada na projektu i kako su riješeni.

Problem 1: App container se podiže prije MySQL-a

Opis: App container je pokušavao spojiti na bazu podataka pri pokretanju, ali MySQL još nije bio spreman, što je rezultiralo greškom veze i padom containera.

Rješenje: Dodan je health check MySQL servisu u podman-compose.yml (mysqladmin ping) i postavljen je depends_on s condition: service_healthy na app servisu. Time je osigurano da se app pokreće tek nakon što MySQL prihvaća veze.

Problem 2: pip install pao za PyMySQL

Opis: Gradnja imagea je pala jer MySQL C klijentska biblioteka nije bila prisutna u slim baznom imageu.

Rješenje: Dodano je apt-get install -y default-libmysqlclient-dev gcc pkg-config u Dockerfile prije pip install koraka.

Problem 3: Integracijski testovi padaju – No module named flask

Opis: Pytest nije mogao uvesti Flask jer na hostu (CentOS VM) Flask nije bio instaliran – bio je dostupan samo unutar Docker imagea.

Rješenje: Flask i sve ovisnosti instalirane su direktno na hostu: pip3 install flask flask-wtf flask-sqlalchemy pymysql cryptography wtforms.

Problem 4: Integracijski testovi padaju – MySQL Connection Refused

Opis: Pytest je pokušavao spojiti se na MySQL na 127.0.0.1:3306, ali MySQL container nije eksponirao port na host (vidljivo iz praznog outputa naredbe podman port tempconverter-db).

Rješenje: Za lokalni deployment dodan je port mapping 3306:3306 za db servis u podman-compose.yml. Integracijski testovi naknadno su prebačeni na privremenu SQLite bazu (postavljenu kroz DATABASE_URL prije importa aplikacije) kako ne bi ovisili o pokrenutom MySQL-u; time je ujedno riješeno zaglavljivanje CI workflowa koje je nastajalo zbog MySQL metadata locka pri db.drop_all() na kraju test modula.

Problem 5: cri-dockerd servis ne može se pokrenuti – status 203/EXEC

Opis: Systemd nije mogao pokrenuti cri-dockerd binary unatoč tome što je binary bio ispravan i ručno izvršiv. Greška: exit-code 203/EXEC.

Rješenje: Uzrok je bio SELinux u Enforcing modu koji je blokirao systemd izvođenje binary-ja koji nije imao ispravne SELinux kontekste. Rješenje: setenforce 0 (isključivanje SELinux enforcementa) i ponovan pokretanje servisa.

Problem 6: Minikube – lanac ovisnosti

Opis: Pokretanje Minikubea s --driver=none zahtijevalo je niz dependencies koje nisu bile instalirane: conntrack, crictl, cri-dockerd, CNI plugins.

Rješenje: Sve dependencies instalirane su redom prema uputama u error porukama Minikubea. Konačna Minikube naredba: minikube start --driver=none --cri-socket /run/cri-dockerd.sock.

Problem 7: DNS greška u Podman mreži nakon VM resumea

Opis: Nakon suspendiranja i resumeanja VMware Workstation virtualne mašine, aplikacija je prijavila Internal Server Error. U logovima containera vidljiva je greška: `sqlalchemy.exc.OperationalError: Can't connect to MySQL server on 'db' ([Errno -3] Temporary failure in name resolution)`. DNS razrješavanje unutar Podman bridge mreže izgubilo se nakon što je VM izašao iz suspended statea, pa app container nije mogao razriješiti hostname db u IP adresu baze podataka.

Rješenje: Pokretanje naredbe `podman-compose down && podman-compose up -d` kojom se oba containera gasi i ponovo podižu s novom mrežnom konfiguracijom. Nakon ponovnog podizanja DNS razrješavanje je ispravno uspostavljeno i aplikacija je normalno funkcionirala.

11. Zaključak

Ovaj projekt uspješno je demonstrirao cijeli DevOps workflow za kontejnerizaciju i deployment web aplikacije. Počevši od izvornog koda, napisan je Dockerfile koji pakira Flask aplikaciju u reproducibilan, prenosiv container image. Image je pushan na javni registry i automatiziran CI/CD pipeline konfiguriran je za obnavljanje i testiranje imagea pri svakoj promjeni koda.

Aplikacija je deployana lokalno s Podmanom radi provjere komunikacije između app i database containera, a korištenje resursa uspoređeno je s tradicionalnom virtualnom mašinom. Dva container orchestration sustava korištena su za produkcijski stil deploymenta: Docker Swarm zbog jednostavnosti i Kubernetes zbog snage i bogatstva ekosustava.

Praktično iskustvo potvrdilo je da su containeri značajno poboljšanje u odnosu na virtualne mašine za ovu vrstu radnog opterećenja. Container orchestration dodaje self-healing, load balancing i deklarativno upravljanje životnim ciklusom koje bi inače zahtijevalo značajno prilagođeno skriptiranje.

12. Literatura

- Docker Documentation – Dockerfile Reference: <https://docs.docker.com/engine/reference/builder/>
- Kubernetes Documentation – Kubernetes Basics: <https://kubernetes.io/docs/tutorials/kubernetes-basics/>
- Phoenix NAP – Podman Compose Guide: <https://phoenixnap.com/kb/podman-compose>
- Minikube Documentation: <https://minikube.sigs.k8s.io/docs/>
- Flask Documentation: <https://flask.palletsprojects.com/>
- Docker Swarm Mode Overview: <https://docs.docker.com/engine/swarm/>
- cri-dockerd GitHub: <https://github.com/Mirantis/cri-dockerd>
- GitHub jstanesic/tempconverter: <https://github.com/jstanesic/tempconverter>

<https://github.com/lisirac1/intoDevops> - GitHub od autora

Prilog A – Dockerfile

```
FROM python:3.11-slim

RUN apt-get update && apt-get upgrade -y && apt-get install -y \
    default-libmysqlclient-dev gcc pkg-config \
    && apt-get clean && rm -rf /var/lib/apt/lists/*

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "-m", "flask", "run", "--host=0.0.0.0", "--port=5000"]
```

Prilog B – podman-compose.yml

```
version: '3.8'
services:
  db:
    image: mysql:8.0
    container_name: tempconverter-db
    ports:
      - '3306:3306'
    environment:
      MYSQL_ROOT_PASSWORD: rootpassword
      MYSQL_DATABASE: tempconverterdb
      MYSQL_USER: appuser
      MYSQL_PASSWORD: apppassword
    volumes:
      - db_data:/var/lib/mysql
    networks: [tempnet]
    healthcheck:
      test: ['CMD', 'mysqladmin', 'ping', '-h', 'localhost', '-u', 'appuser', '-pappppassword']
      interval: 10s
      timeout: 5s
      retries: 5
  app:
    image: localhost/tempconverter:dev
    container_name: tempconverter-app
    ports: ['5000:5000']
    environment:
      DB_USER: appuser
      DB_PASS: apppassword
      DB_HOST: db
      DB_NAME: tempconverterdb
      STUDENT: 'Lsirac'
      COLLEGE: 'Algebra Bernays University'
    networks: [tempnet]
    depends_on:
      db:
        condition: service_healthy
networks:
  tempnet:
    driver: bridge
volumes:
  db_data:
```

Prilog C – docker-stack.yml (Docker Swarm)

```
version: '3.8'
services:
  db:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: rootpassword
```

```
    MYSQL_DATABASE: tempconverterdb
    MYSQL_USER: appuser
    MYSQL_PASSWORD: apppassword
volumes: [db_data:/var/lib/mysql]
networks: [tempnet]
deploy:
  replicas: 1
  placement:
    constraints: [node.role == manager]
app:
  image: lsirac/tempconverter:latest
  ports:
    - target: 5000
      published: 80
      mode: ingress
  environment:
    DB_USER: appuser
    DB_PASS: apppassword
    DB_HOST: db
    DB_NAME: tempconverterdb
    STUDENT: 'Lsirac'
    COLLEGE: 'Algebra Bernays University'
  networks: [tempnet]
  deploy:
    replicas: 2
    placement:
      max_replicas_per_node: 1
networks:
  tempnet:
    driver: overlay
volumes:
  db_data:
```

Prilog D – Kubernetes manifesti

k8s/secret.yaml

```
apiVersion: v1
kind: Secret
metadata:
  name: tempconverter-secret
type: Opaque
data:
  db-user: YXBwdXNlcmRlcmRi
  db-pass: YXBwcGFzc3dvcmQ=
  db-name: dGVtcGNvbnZlcnRlcmRi
  db-root-pass: cm9vdHBhc3N3b3Jk
```

k8s/pvc.yaml

```
# k8s/pvc.yaml - Persistent storage for MySQL database
# Apply: kubectl apply -f k8s/pvc.yaml

apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mysql-pvc
  namespace: default
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```

k8s/mysql.yaml

```
# k8s/mysql.yaml - MySQL 8 Deployment + ClusterIP Service
```

```
# Apply: kubectl apply -f k8s/mysql.yaml

apiVersion: apps/v1
kind: Deployment
metadata:
  name: mysql
  namespace: default
  labels:
    app: mysql
spec:
  replicas: 1                                # Exactly one DB instance (requirement)
  selector:
    matchLabels:
      app: mysql
  template:
    metadata:
      labels:
        app: mysql
    spec:
      containers:
        - name: mysql
          image: mysql:8.0
          ports:
            - containerPort: 3306
          env:
            - name: MYSQL_ROOT_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: tempconverter-secret
                  key: db-root-pass
            - name: MYSQL_DATABASE
              valueFrom:
                secretKeyRef:
                  name: tempconverter-secret
                  key: db-name
            - name: MYSQL_USER
              valueFrom:
                secretKeyRef:
                  name: tempconverter-secret
                  key: db-user
            - name: MYSQL_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: tempconverter-secret
                  key: db-pass
      volumeMounts:
        - name: mysql-storage
          mountPath: /var/lib/mysql
      readinessProbe:
        exec:
          command: ["mysqladmin", "ping", "-h", "localhost"]
          initialDelaySeconds: 20
          periodSeconds: 10
      volumes:
        - name: mysql-storage
          persistentVolumeClaim:
            claimName: mysql-pvc

---
apiVersion: v1
kind: Service
metadata:
  name: mysql
  namespace: default
spec:
  selector:
    app: mysql
  ports:
    - port: 3306
      targetPort: 3306
  clusterIP: None    # Headless service – stable DNS name for the app to connect to
```

k8s/app.yaml

```
# k8s/app.yaml - TempConverter Deployment + Service (NodePort) + Ingress
# Apply: kubectl apply -f k8s/app.yaml
#
# Skaliranje na 3 replike:
# kubectl scale deployment tempconverter --replicas=3

apiVersion: apps/v1
kind: Deployment
metadata:
  name: tempconverter
  namespace: default
  labels:
    app: tempconverter
spec:
  replicas: 2                                # Dvije replike aplikacije (zahtjev 6b)
  selector:
    matchLabels:
      app: tempconverter
  template:
    metadata:
      labels:
        app: tempconverter
    spec:
      # Anti-affinity: Kubernetes preferira rasporediti replike na razlicite
      # nodeove. Koristi se 'preferred' (a ne 'required') kako bi na lokalnom
      # single-node minikube clusteru sve replike i dalje radile (Running),
      # dok bi se u produkcijskom multi-node clusteru rasporedile na razlicite
      # nodeove (zahtjev 6b: replike ne na istom nodeu).
      affinity:
        podAntiAffinity:
          preferredDuringSchedulingIgnoredDuringExecution:
            - weight: 100
              podAffinityTerm:
                labelSelector:
                  matchExpressions:
                    - key: app
                      operator: In
                      values:
                        - tempconverter
                topologyKey: "kubernetes.io/hostname"
    containers:
      - name: tempconverter
        image: lsirac/tempconverter:latest
        ports:
          - containerPort: 5000
        env:
          - name: DB_HOST
            value: "mysql"                    # Naziv MySQL Servicea
          - name: DB_USER
            valueFrom:
              secretKeyRef:
                name: tempconverter-secret
                key: db-user
          - name: DB_PASS
            valueFrom:
              secretKeyRef:
                name: tempconverter-secret
                key: db-pass
          - name: DB_NAME
            valueFrom:
              secretKeyRef:
                name: tempconverter-secret
                key: db-name
          - name: STUDENT
            value: "Lsirac"
          - name: COLLEGE
            value: "Algebra Bernays University"
```

```
- name: FLASK_APP
  value: "app.py"
readinessProbe:
  httpGet:
    path: /
    port: 5000
  initialDelaySeconds: 15
  periodSeconds: 10
livenessProbe:
  httpGet:
    path: /
    port: 5000
  initialDelaySeconds: 30
  periodSeconds: 30

---
# NodePort Service - eksponira aplikaciju lokalno na minikubeu (port 31041).
# Interni port Servicea je 80, preusmjeren na containerPort 5000.
apiVersion: v1
kind: Service
metadata:
  name: tempconverter-svc
  namespace: default
spec:
  selector:
    app: tempconverter
  ports:
    - port: 80
      targetPort: 5000
      nodePort: 31041
  type: NodePort

---
# Ingress - produkcijsko izlaganje aplikacije vanjskom svijetu preko porta 80
# (zahtjev 6a). Zahtijeva instaliran Ingress Controller (npr. nginx-ingress).
# Na lokalnom minikubeu koristi se NodePort (gore), dok Ingress predstavlja
# produkcijski put izlaganja preko standardnog porta 80.
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: tempconverter-ingress
  namespace: default
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  rules:
    - host: tempconverter.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: tempconverter-svc
                port:
                  number: 80
```

Prilog E – GitHub Actions CI/CD pipeline (.github/workflows/ci.yml)

```
name: CI - Test & Build

on:
  push:
    branches: [ "main", "dev" ]
  pull_request:
    branches: [ "main" ]

jobs:
```

```
test:
  name: Unit & Integration Tests
  runs-on: ubuntu-latest
  timeout-minutes: 10           # safety net: fail fast instead of hanging

  # No MySQL service needed: the tests run against an in-memory/file SQLite
  # database (the test fixture sets DATABASE_URL to SQLite before importing
  # the app). Production still uses MySQL via DB_* env vars.
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-python@v5
      with: { python-version: '3.11' }
    - run: pip install -r requirements.txt pytest pytest-cov
    - run: python -m pytest tests/ -v --tb=short

build:
  name: Build & Push Container Image
  runs-on: ubuntu-latest
  needs: test    # only build if tests pass
  timeout-minutes: 15

  steps:
    - name: Checkout source code
      uses: actions/checkout@v4

    - name: Log in to Docker Hub
      uses: docker/login-action@v3
      with:
        username: ${ secrets.DOCKERHUB_USERNAME }
        password: ${ secrets.DOCKERHUB_TOKEN }

    - name: Set image tag based on branch
      id: tag
      run: |
        if [ "${ github.ref }" == "refs/heads/main" ]; then
          echo "tag=latest" >> $GITHUB_OUTPUT
        else
          echo "tag=dev" >> $GITHUB_OUTPUT
        fi

    - name: Build and push image
      uses: docker/build-push-action@v5
      with:
        context: .
        push: true
        tags: |
          ${ secrets.DOCKERHUB_USERNAME }/tempconverter:${ steps.tag.outputs.tag }
```